

Российский университет дружбы народов

Факультет физико-математических и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №8

Дисциплина: Архитектура компьютеров

Студент: Алексеев Тимофей

Группа: НКАбд-07-25

Москва

2025 г.

Page 1

Содержание

1. Цель работы	5
2. Задание	6
3. Теоретическое введение	7
4. Выполнение лабораторной работы	8
4.1 Реализация циклов в NASM	8
4.2 Обработка аргументов командной строки	13
4.3 Задание для самостоятельной работы	17
5. Выводы	20
6. Список литературы	21

Список иллюстраций

4.1 Создание каталога	8
4.2 Копирование программы из листинга	9
4.3 Запуск программы	9
4.4 Изменение программы	10
4.5 Запуск измененной программы	11
4.6 Добавление push и pop в цикл программы	12
4.7 Запуск измененной программы	13
4.8 Копирование программы из листинга	13
4.9 Запуск второй программы	14
4.10 Копирование программы из третьего листинга	15
4.11 Запуск третьей программы	15
4.12 Изменение третьей программы	16
4.13 Запуск измененной третьей программы	16
4.14 Написание программы для самостоятельной работы	17
4.15 Запуск программы для самостоятельной работы	19

Список таблиц

1. Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2. Задание

1. Реализация циклом в NASM
2. Обработка аргументов командной строки
3. Самостоятельное написание программы по материалам лабораторной работы

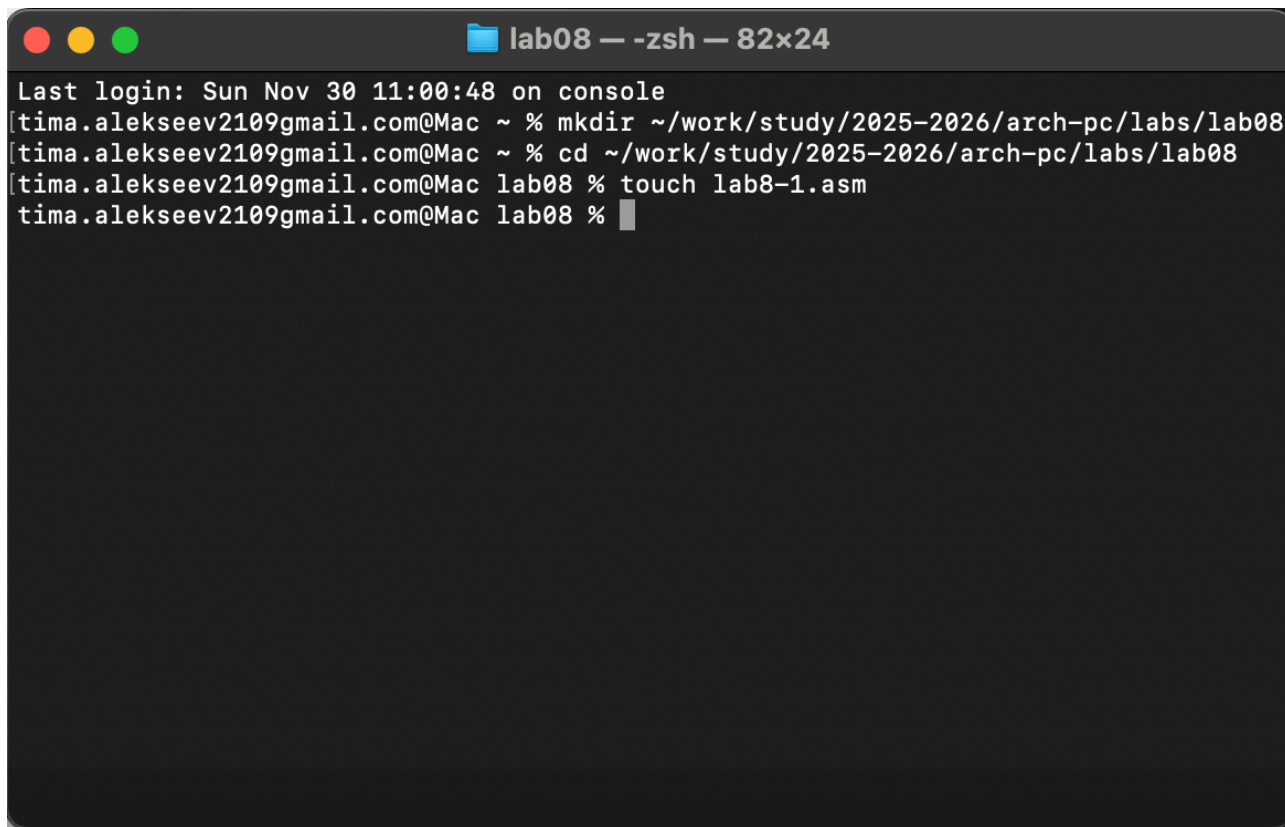
3. Теоретическое введение

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл — первым ушёл»). Стек является частью архитектуры процессора и реализован на аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды. Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров.

4. Выполнение лабораторной работы

4.1 Реализация циклов в NASM

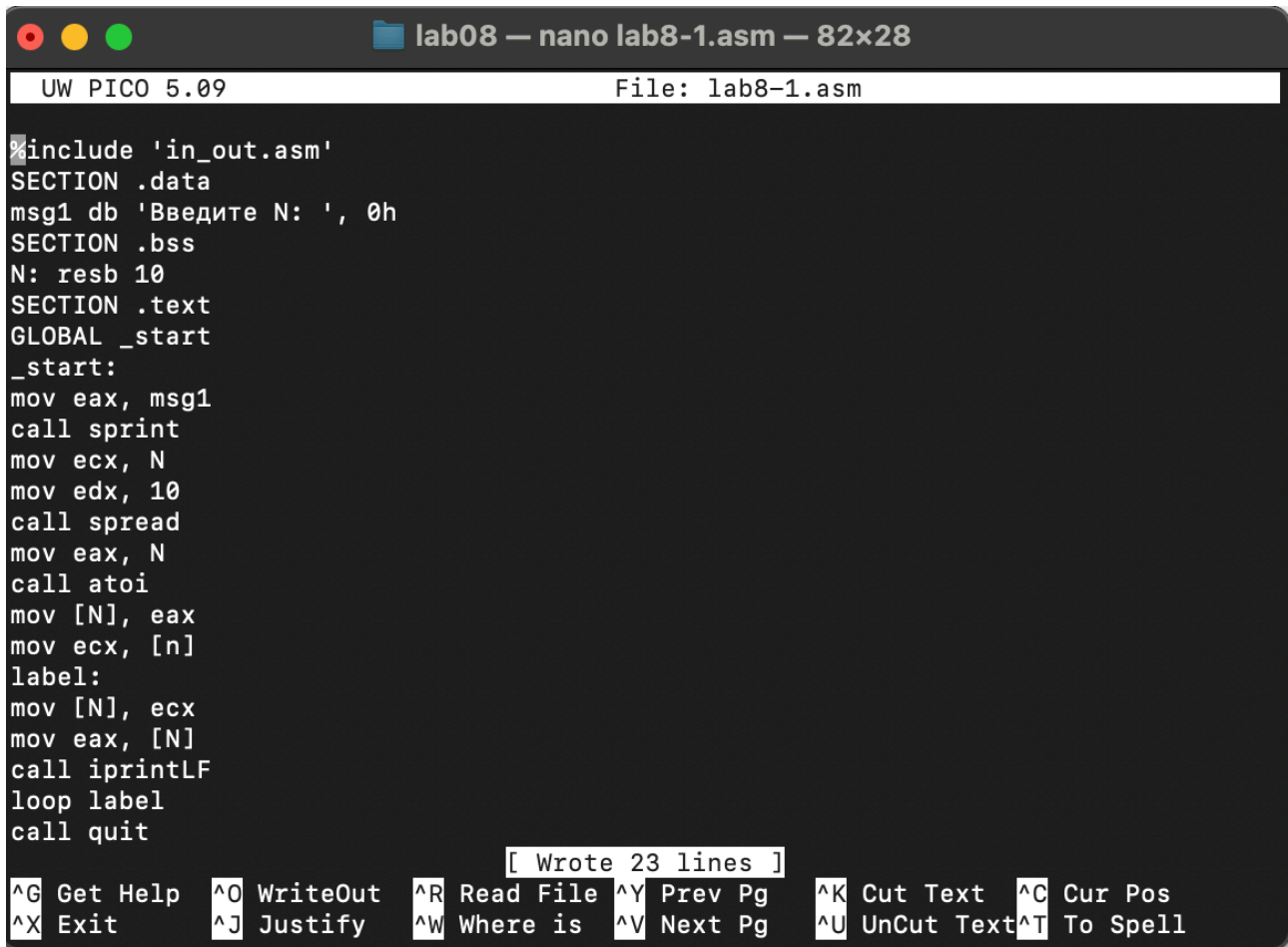
Создаю каталог для программ лабораторной работы №8 (рис. 4.1).

A screenshot of a terminal window titled "lab08 — -zsh — 82x24". The terminal shows the following commands and output:

```
Last login: Sun Nov 30 11:00:48 on console
[tima.alekseev2109gmail.com@Mac ~ % mkdir ~/work/study/2025-2026/arch-pc/labs/lab08]
[tima.alekseev2109gmail.com@Mac ~ % cd ~/work/study/2025-2026/arch-pc/labs/lab08 ]
[tima.alekseev2109gmail.com@Mac lab08 % touch lab8-1.asm
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.1: Создание каталога

Копирую в созданный файл программу из листинга. (рис. 4.2).



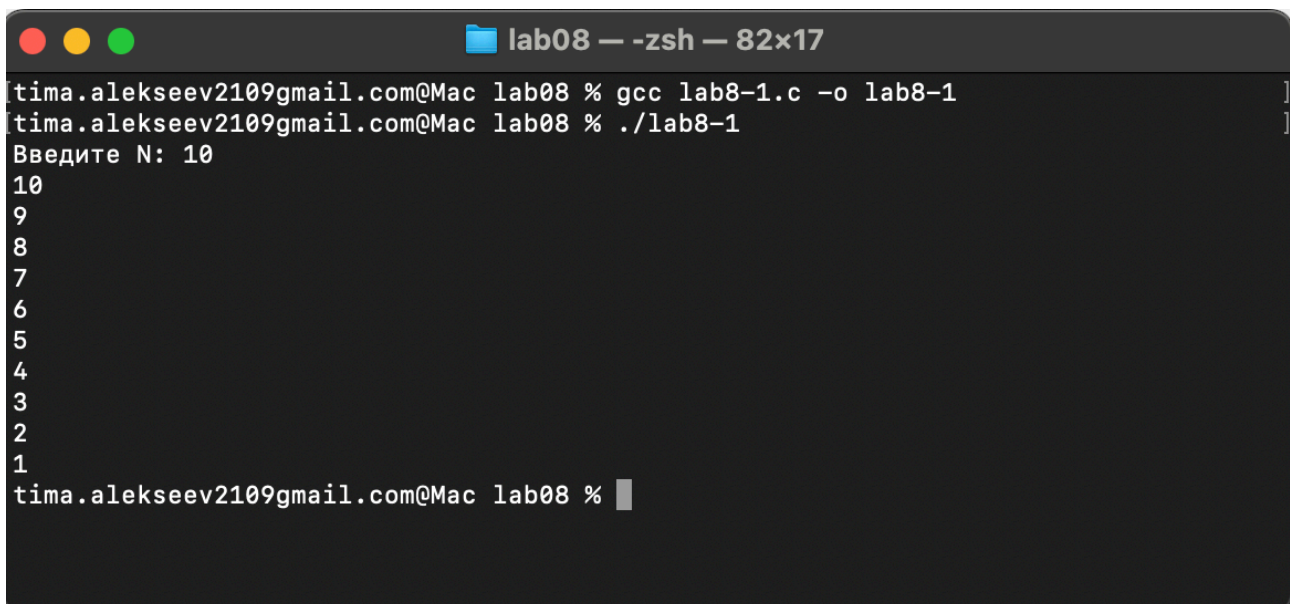
```
lab08 — nano lab8-1.asm — 82x28
UW PICO 5.09 File: lab8-1.asm

%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ', 0h
SECTION .bss
N: resb 10
SECTION .text
GLOBAL _start
_start:
mov eax, msg1
call sprint
mov ecx, N
mov edx, 10
call spread
mov eax, N
call atoi
mov [N], eax
mov ecx, [n]
label:
mov [N], ecx
mov eax, [N]
call iprintLF
loop label
call quit
```

[Wrote 23 lines]

^G Get Help **^O** WriteOut **^R** Read File **^Y** Prev Pg **^K** Cut Text **^C** Cur Pos
^X Exit **^J** Justify **^W** Where is **^V** Next Pg **^U** UnCut Text **^T** To Spell

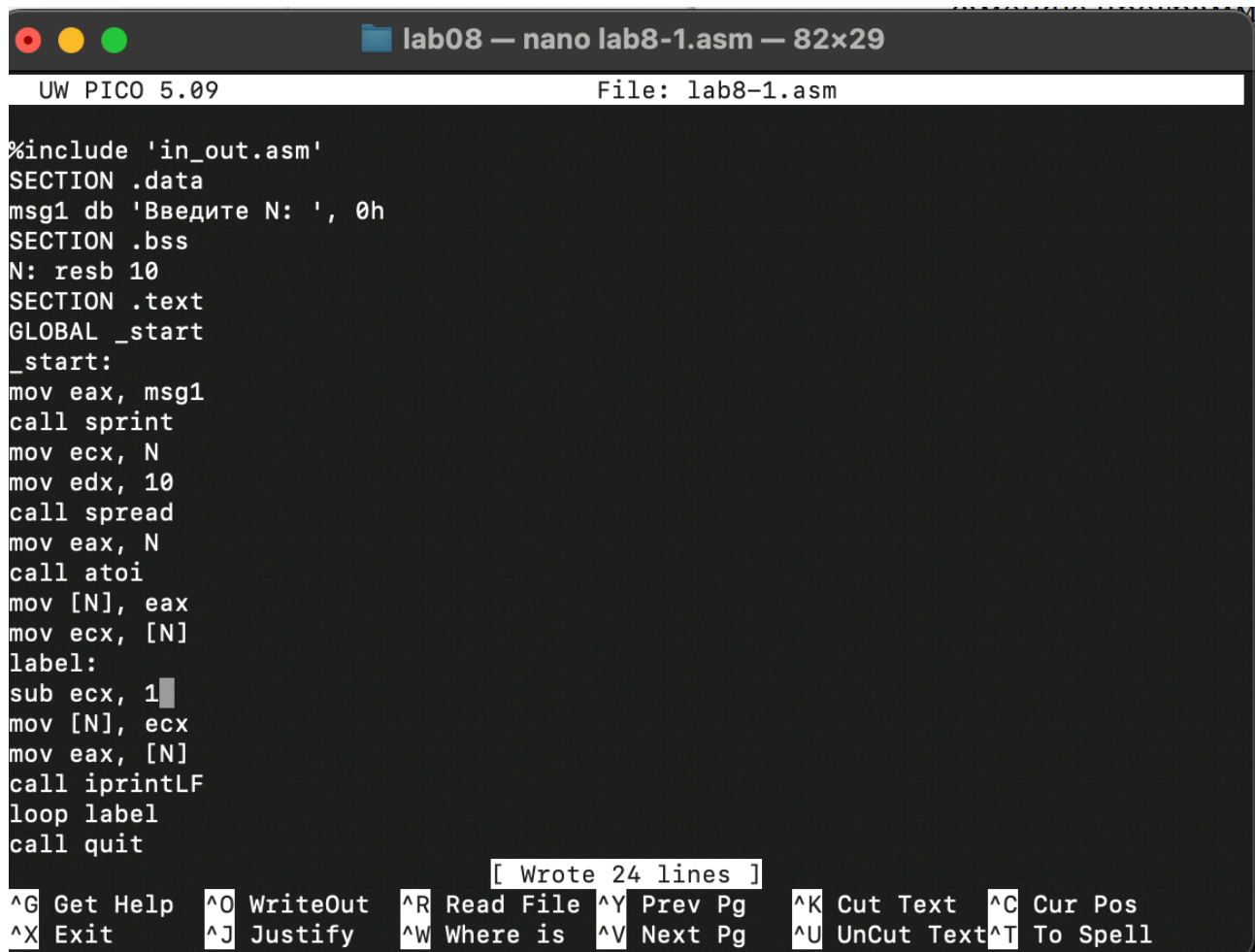
Рис. 4.2: Копирование программы из листинга
Запускаю программу, она показывает работу циклов в NASM (рис. 4.3).



```
lab08 — -zsh — 82x17
tima.alekseev2109gmail.com@Mac lab08 % gcc lab8-1.c -o lab8-1
tima.alekseev2109gmail.com@Mac lab08 % ./lab8-1
Введите N: 10
10
9
8
7
6
5
4
3
2
1
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.3: Запуск программы

Заменяю программу изначальную так, что в теле цикла я изменяю значение регистра ecx (рис. 4.4).



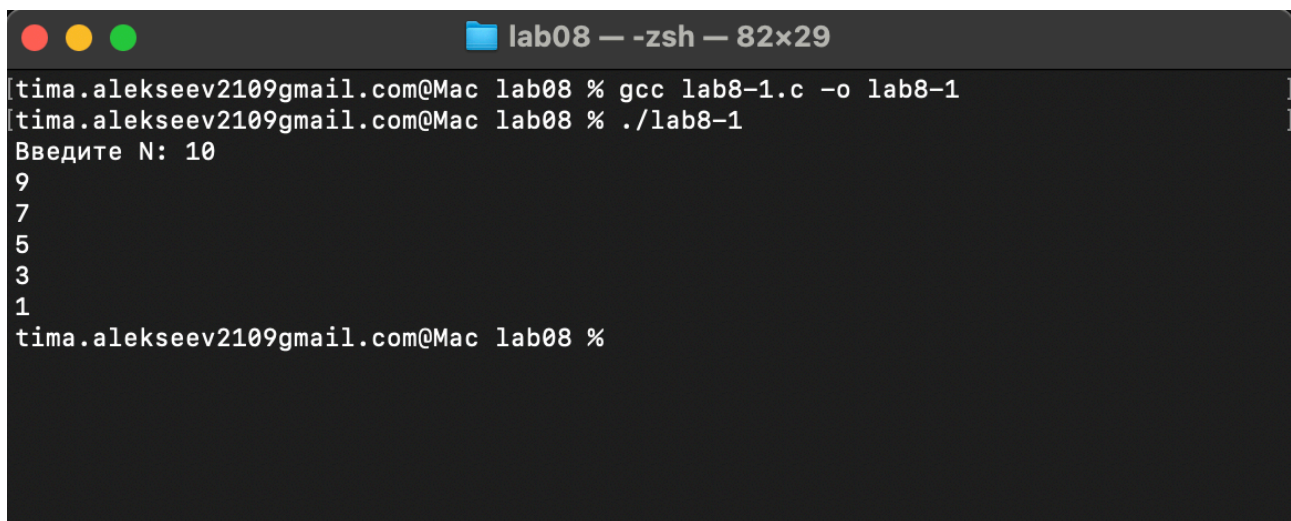
```
%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ', 0h
SECTION .bss
N: resb 10
SECTION .text
GLOBAL _start
_start:
mov eax, msg1
call sprint
mov ecx, N
mov edx, 10
call spread
mov eax, N
call atoi
mov [N], eax
mov ecx, [N]
label:
sub ecx, 1
mov [N], ecx
mov eax, [N]
call iprintLF
loop label
call quit
```

[Wrote 24 lines]

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

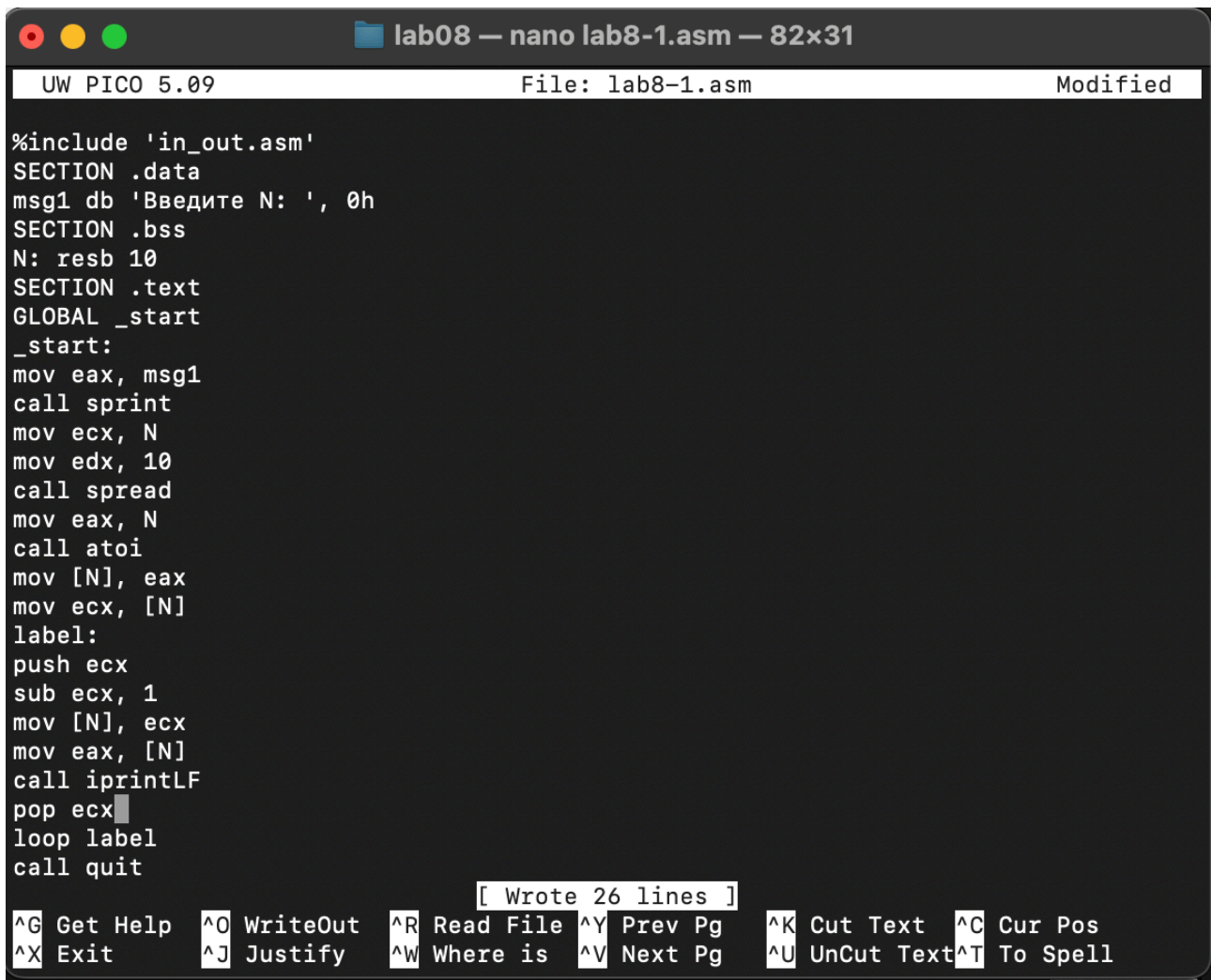
Рис. 4.4: Изменение программы

Из-за того, что теперь регистр ecx на каждой итерации уменьшается на 2 значения, количество итераций уменьшается вдвое (рис. 4.5).



```
lab08 — -zsh — 82x29
tima.alekseev2109gmail.com@Mac lab08 % gcc lab8-1.c -o lab8-1
tima.alekseev2109gmail.com@Mac lab08 % ./lab8-1
Введите N: 10
9
7
5
3
1
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.5: Запуск измененной программы
Добавляю команды push и pop в программу (рис. 4.6).



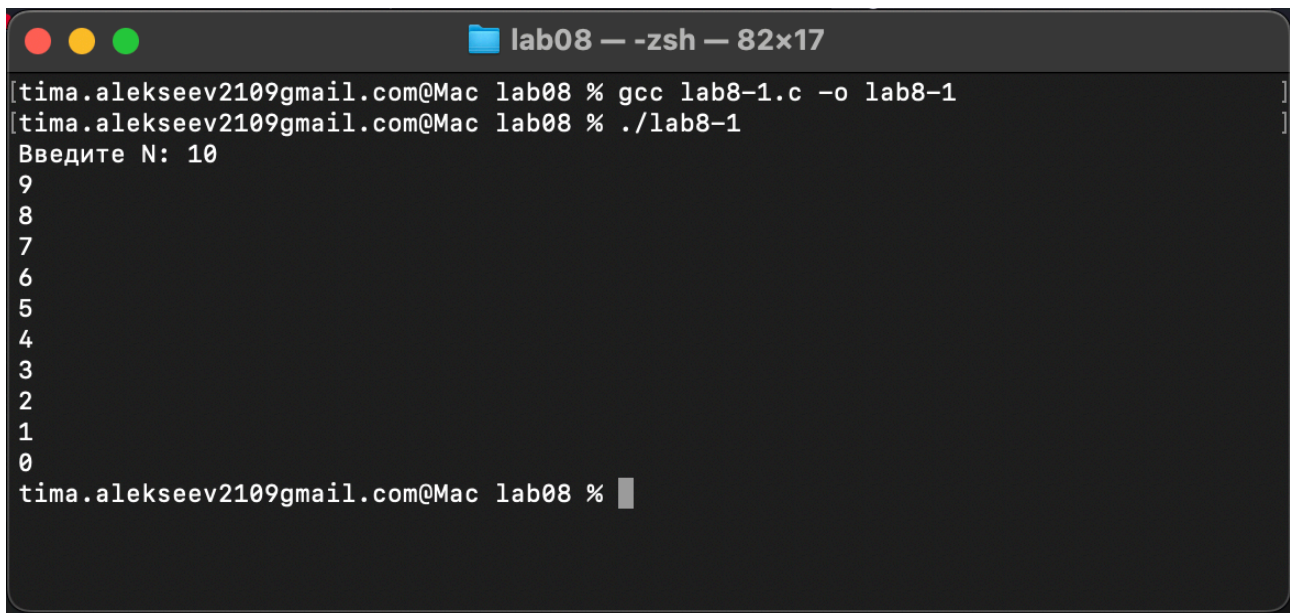
```
lab08 — nano lab8-1.asm — 82x31
UW PICO 5.09 File: lab8-1.asm Modified

#include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ', 0h
SECTION .bss
N: resb 10
SECTION .text
GLOBAL _start
_start:
mov eax, msg1
call sprint
mov ecx, N
mov edx, 10
call spread
mov eax, N
call atoi
mov [N], eax
mov ecx, [N]
label:
push ecx
sub ecx, 1
mov [N], ecx
mov eax, [N]
call iprintLF
pop ecx
loop label
call quit
```

[Wrote 26 lines]

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text	^C Cur Pos
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Text	^T To Spell

Рис. 4.6: Добавление push и pop в цикл программы
Теперь количество итераций совпадает введенному N, но произошло смещение выводимых чисел на -1 (рис. 4.7).



A terminal window titled "lab08 — -zsh — 82x17". The prompt is "tima.alekseev2109gmail.com@Mac lab08 %". The user enters "gcc lab8-1.c -o lab8-1" and then "./lab8-1". The program prompts "Введите N: 10" and the user enters "9". The program then prints numbers from 9 down to 0. The prompt returns to "tima.alekseev2109gmail.com@Mac lab08 %".

```
tima.alekseev2109gmail.com@Mac lab08 % gcc lab8-1.c -o lab8-1
tima.alekseev2109gmail.com@Mac lab08 % ./lab8-1
Введите N: 10
9
8
7
6
5
4
3
2
1
0
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.7: Запуск измененной программы

4.2 Обработка аргументов командной строки

Создаю новый файл для программы и копирую в него код из следующего листинга (рис. 4.8).



A Nano editor window titled "lab08 — nano lab8-2.asm — 82x20". The status bar shows "UW PICO 5.09" and "File: lab8-2.asm". The code is as follows:

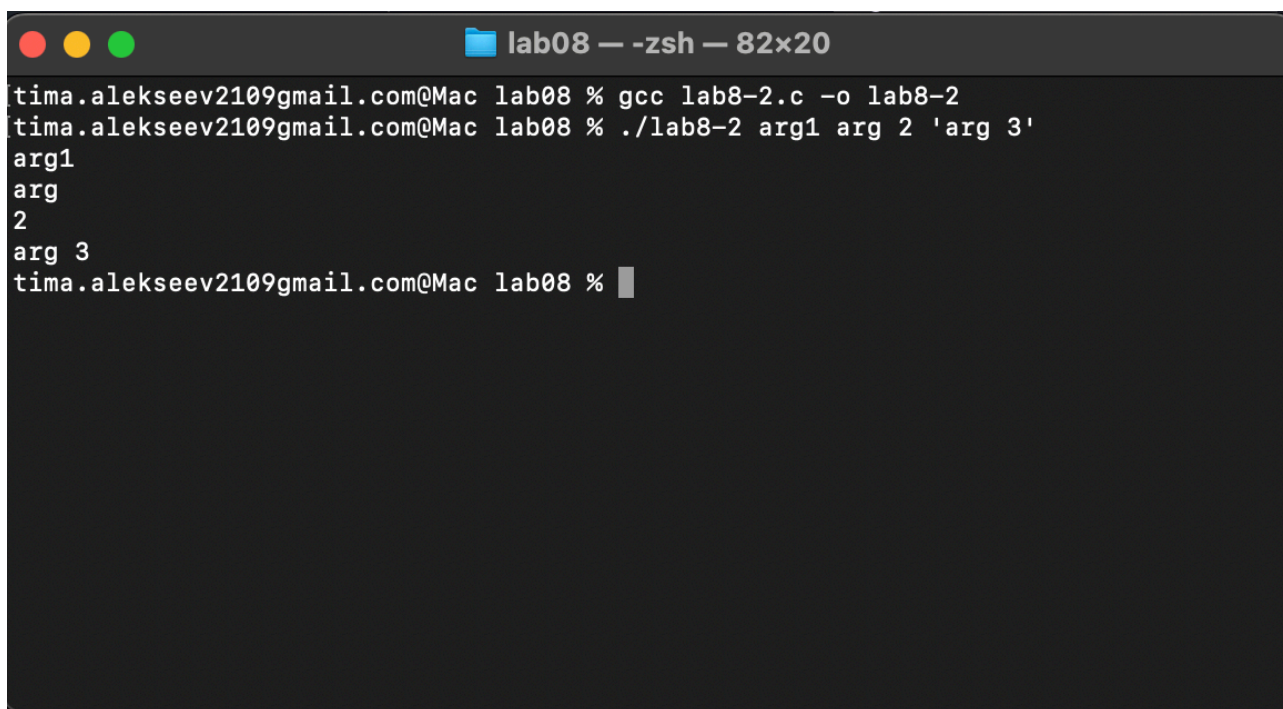
```
%include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
pop ecx
pop edx
sub ecx, 1
next:
cmp ecx, 0
jz _end
pop eax
call sprintLF
loop next
_end:
call quit
```

The status bar also shows "[Wrote 15 lines]" and a list of keyboard shortcuts: ^G Get Help, ^O WriteOut, ^R Read File, ^Y Prev Pg, ^K Cut Text, ^C Cur Pos, ^X Exit, ^J Justify, ^W Where is, ^V Next Pg, ^U UnCut Text, ^T To Spell.

Рис. 4.8: Копирование программы из листинга

Компилирую программу и запускаю, указав аргументы. Программой было

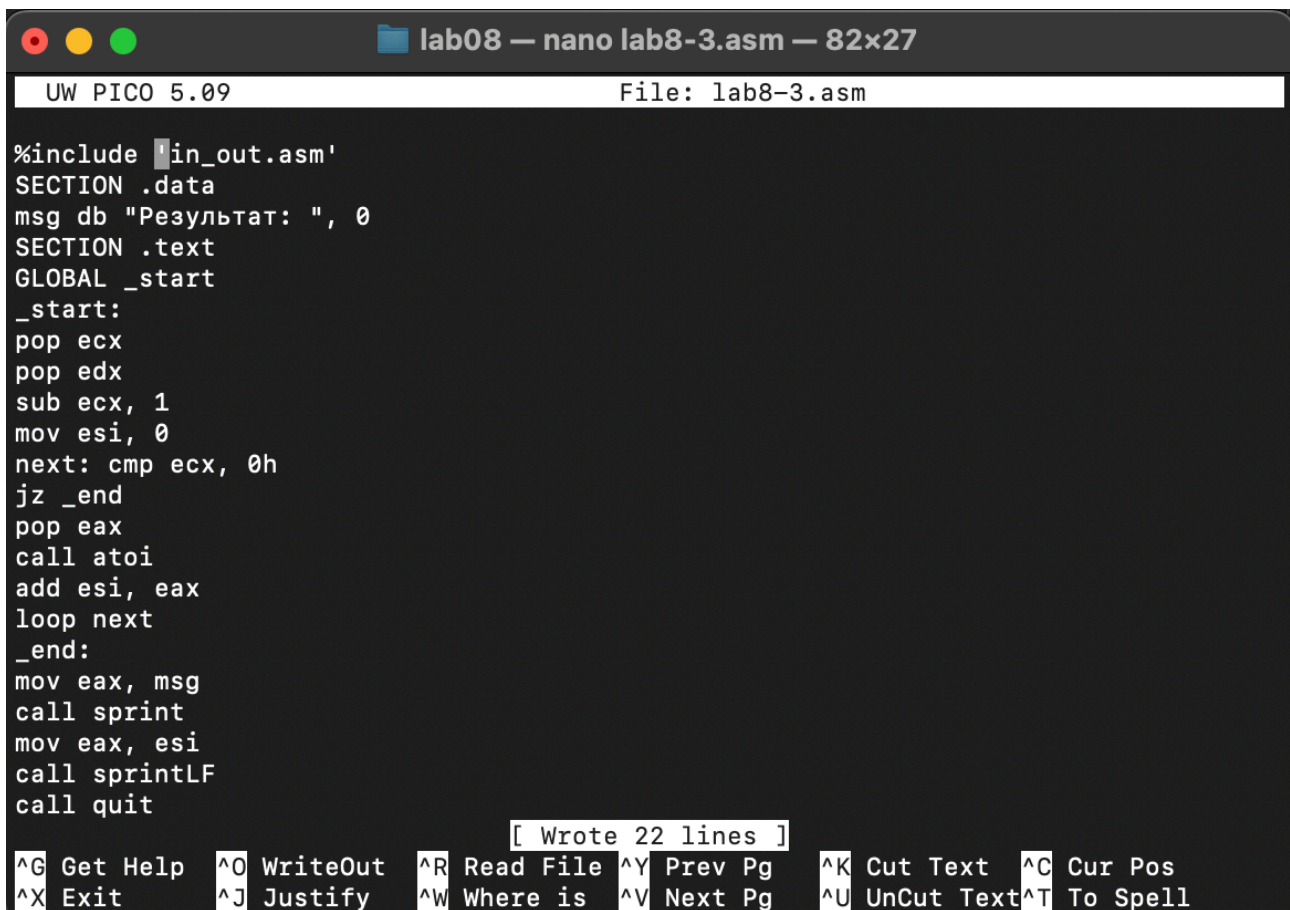
обратно то же количество аргументов, что и было введено (рис. 4.9).

A terminal window titled 'lab08 — -zsh — 82x20' is shown. The prompt is 'tima.alekseev2109gmail.com@Mac lab08 %'. The user enters 'gcc lab8-2.c -o lab8-2'. The prompt changes to 'tima.alekseev2109gmail.com@Mac lab08 %'. The user enters './lab8-2 arg1 arg 2 'arg 3''. The program outputs 'arg1', 'arg', '2', and 'arg 3' on separate lines. The prompt returns to 'tima.alekseev2109gmail.com@Mac lab08 %' with a cursor.

```
tima.alekseev2109gmail.com@Mac lab08 % gcc lab8-2.c -o lab8-2
tima.alekseev2109gmail.com@Mac lab08 % ./lab8-2 arg1 arg 2 'arg 3'
arg1
arg
2
arg 3
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.9: Запуск второй программы

Создаю новый файл для программы и копирую в него код из третьего листинга (рис. 4.10).

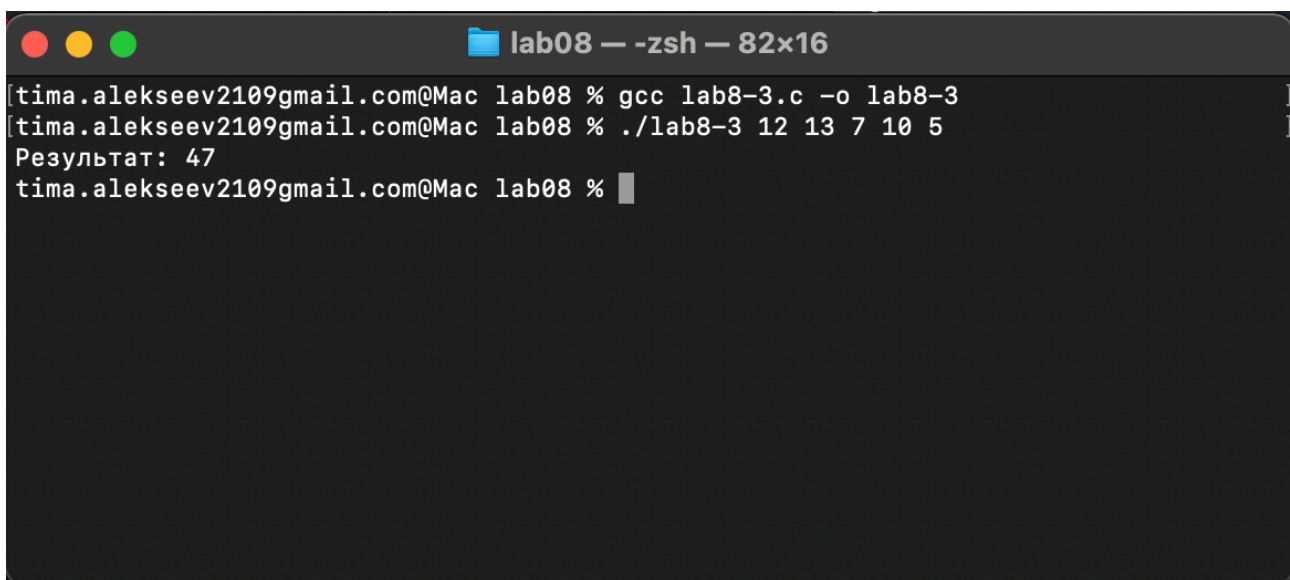


The screenshot shows a nano editor window titled "lab08 — nano lab8-3.asm — 82x27". The status bar at the top indicates "UW PICO 5.09" and "File: lab8-3.asm". The editor contains the following assembly code:

```
%include 'in_out.asm'
SECTION .data
msg db "Результат: ", 0
SECTION .text
GLOBAL _start
_start:
pop ecx
pop edx
sub ecx, 1
mov esi, 0
next: cmp ecx, 0h
jz _end
pop eax
call atoi
add esi, eax
loop next
_end:
mov eax, msg
call sprint
mov eax, esi
call sprintf
call quit
```

At the bottom of the editor, a status bar shows "[Wrote 22 lines]" and a set of keyboard shortcuts: ^G Get Help, ^O WriteOut, ^R Read File, ^Y Prev Pg, ^K Cut Text, ^C Cur Pos, ^X Exit, ^J Justify, ^W Where is, ^V Next Pg, ^U UnCut Text, ^T To Spell.

Рис. 4.10: Копирование программы из третьего листинга
Компилирую программу и запускаю, указав в качестве аргументов некоторые числа, программа их складывает (рис. 4.11).



The screenshot shows a terminal window titled "lab08 — -zsh — 82x16". The prompt is "tima.alekseev2109gmail.com@Mac lab08 %". The user enters the command "gcc lab8-3.c -o lab8-3", and the prompt changes to "tima.alekseev2109gmail.com@Mac lab08 %". The user then enters the command "./lab8-3 12 13 7 10 5", and the output "Результат: 47" is displayed. The prompt returns to "tima.alekseev2109gmail.com@Mac lab08 %".

Рис. 4.11: Запуск третьей программы

Изменяю поведение программы так, чтобы указанные аргументы она умножала, а не складывала (рис. 4.12).



```
lab08 — nano lab8-3.asm — 82x29
UW PICO 5.09 File: lab8-3.asm

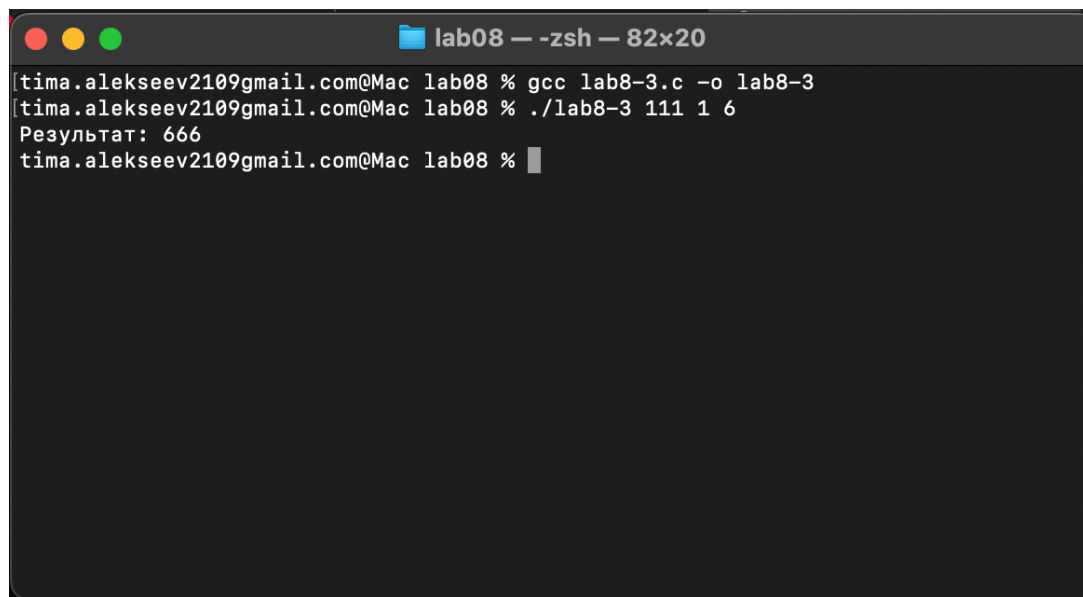
#include 'inout.asm'
SECTION .data
msg db "Результат: ", 0
SECTION .text
GLOBAL _start
_start:
pop ecx
pop edx
sub ecx, 1
mov esi, 1
next:
cmp ecx, 0h
jz _end
pop eax
call atoi
mul esi
mov esi, eax
loop next
_end:
mov eax, msg
call sprint
mov eax, esi
call sprintLF
call quit
```

[Wrote 24 lines]

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

Рис. 4.12: Изменение третьей программы

Программа действительно теперь умножает данные на вход числа (рис. 4.13).

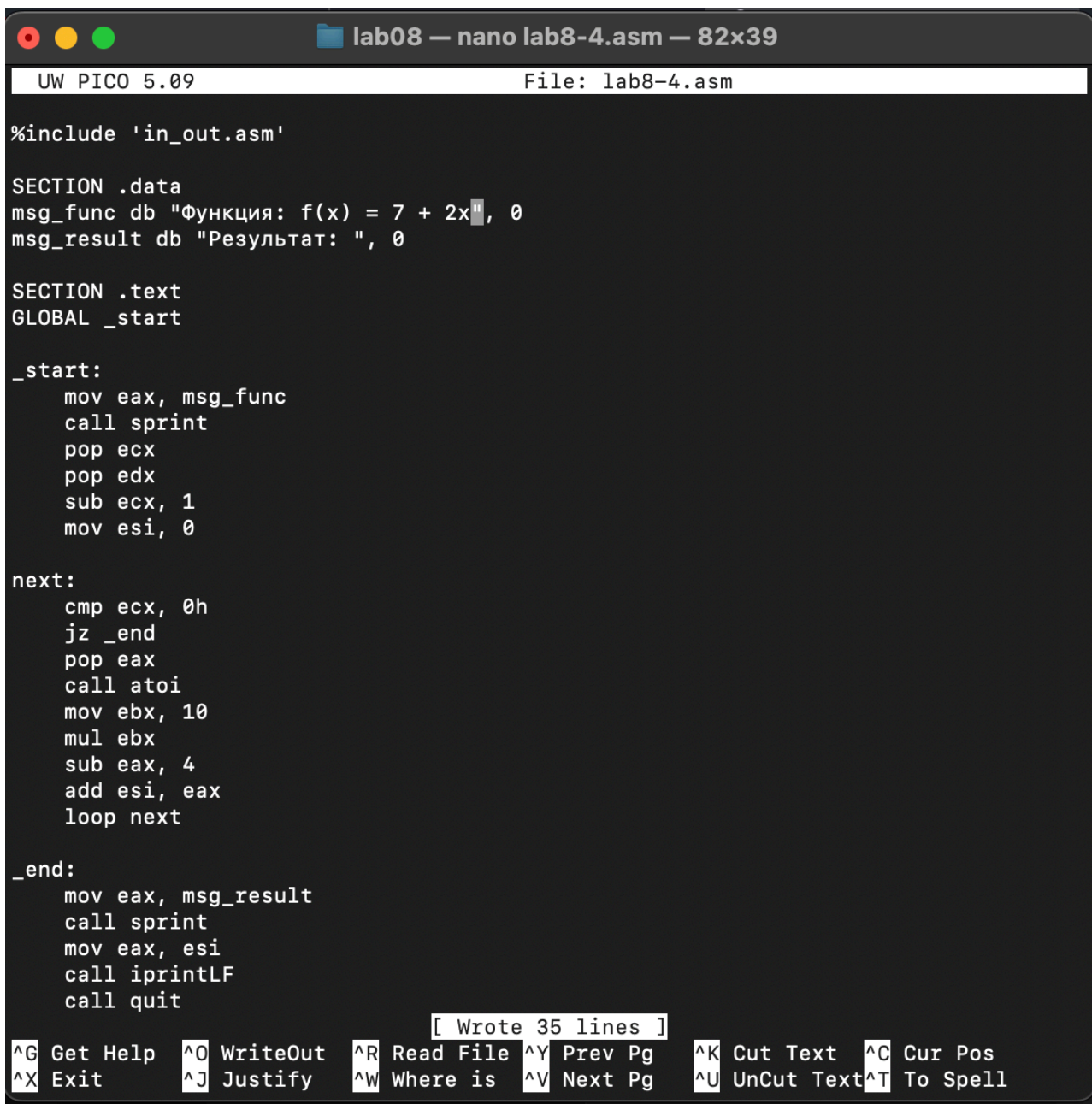


```
lab08 — -zsh — 82x20
tima.alekseev2109gmail.com@Mac lab08 % gcc lab8-3.c -o lab8-3
tima.alekseev2109gmail.com@Mac lab08 % ./lab8-3 111 1 6
Результат: 666
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.13: Запуск измененной третьей программы

4.3 Задание для самостоятельной работы

Пишу программу, которая будет находить сумму значений для функции $f(x) = 7 + 2x$, которая совпадает с моим восьмым вариантом (рис. 4.14).



```
%include 'in_out.asm'

SECTION .data
msg_func db "Функция: f(x) = 7 + 2x", 0
msg_result db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
    mov eax, msg_func
    call sprint
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0

next:
    cmp ecx, 0h
    jz _end
    pop eax
    call atoi
    mov ebx, 10
    mul ebx
    sub eax, 4
    add esi, eax
    loop next

_end:
    mov eax, msg_result
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

[Wrote 35 lines]

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text	^C Cur Pos
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Text	^T To Spell

Рис. 4.14: Написание программы для самостоятельной работы

Код программы:

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg_func db "Функция: f(x) = 10x - 4", 0
```

```
msg_result db "Результат: ", 0
```

```
SECTION .text
```

```
GLOBAL _start
```

```

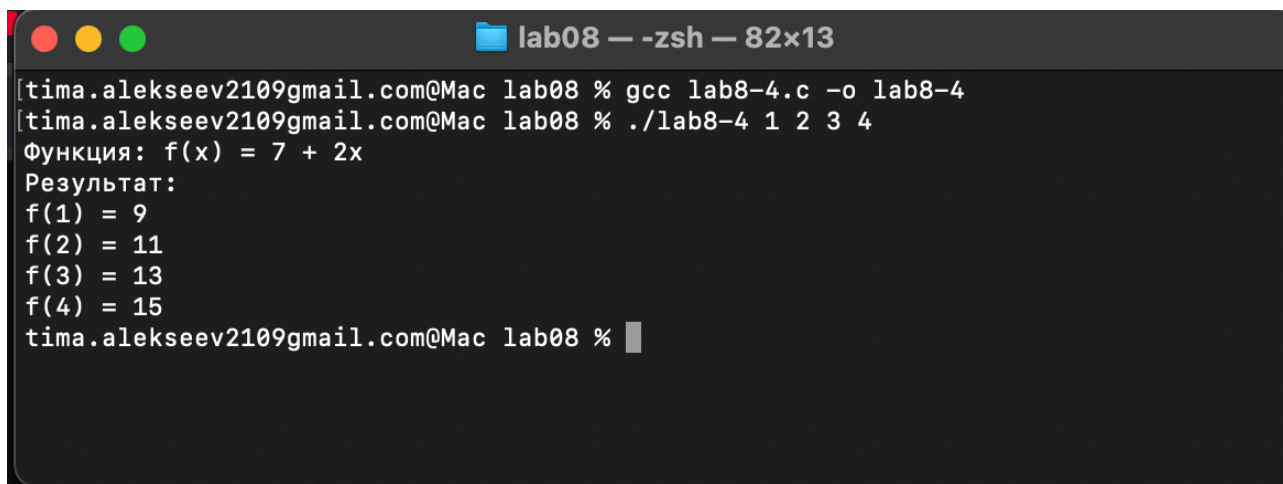
_start:
    mov eax, msg_func
    call sprint
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0

next:
    cmp ecx, 0h
    jz _end
    pop eax
    call atoi
    mov ebx, 10
    mul ebx
    sub eax, 4
    add esi, eax
    loop next

_end:
    mov eax, msg_result
    call sprint
    mov eax, esi
    call iprintLF
    call quit

```

Проверяю работу программы, указав в качестве аргумента несколько чисел (рис. 4.15).



A terminal window titled "lab08 — -zsh — 82x13" with standard macOS window controls (red, yellow, green buttons). The terminal shows the following sequence of commands and output:

```
[tima.alekseev2109gmail.com@Mac lab08 % gcc lab8-4.c -o lab8-4  
[tima.alekseev2109gmail.com@Mac lab08 % ./lab8-4 1 2 3 4  
Функция:  $f(x) = 7 + 2x$   
Результат:  
f(1) = 9  
f(2) = 11  
f(3) = 13  
f(4) = 15  
tima.alekseev2109gmail.com@Mac lab08 %
```

Рис. 4.15: Запуск программы для самостоятельной работы

5. Выводы

В результате выполнения данной лабораторной работы я приобрел навыки написания программ с использованием циклов а также научился обрабатывать аргументы командной строки.

6. Список литературы

1. Курс на ТУИС
2. Лабораторная работа №8
3. Программирование на языке ассемблера NASM Столяров А. В.